



БЕЗОПАСНОСТЬ СЕТЕЙ И ОБЛАЧНЫХ СРЕД

Модуль 1. ТЕХНОЛОГИИ ВИРТУАЛИЗАЦИИ И КОНТЕЙНЕРИЗАЦИИ НА ПРИМЕРЕ DOCKER/K8S

ПРАКТИЧЕСКОЕ ЗАДАНИЕ #02

Выполнил: Юрий Шамрай (MIFIIB/1-й поток)

УСЛОВИЕ ПРАКТИЧЕСКОГО ЗАДАНИЯ:

Разверните веб-приложение на кластере Minikube.

1. В качестве образов для двух контейнеров можно (но не обязательно) использовать уже созданные в модулях по Docker образы.
2. В файле index.php нужно чтобы логин/пароль должны браться из переменных окружения.
3. Если используете публичные образы, проследите, чтобы образ веб-приложения брал данные для подключения к БД из переменных окружения.
4. Необходимо сделать таблицу (например, в формате Excel) с инвентаризацией кластера. В кратком виде указать установленное ПО на нодах, количество нод, ОС на нодах, адреса.

УСЛОВИЯ РЕАЛИЗАЦИИ:

В качестве ответа предоставьте:

все конфигурационные файлы,
скриншот из браузера с работающим веб-приложением,
вывод команды `kubectl get all -o wide`,
табличку с инвентаризацией.

Все результаты необходимо загрузить на свой GitHub и прикрепить ссылку на репозиторий в качестве ответа.

Практическое задание выполнено на виртуальной машине, с установленной Ubuntu 23.04 (Lunar Lobster).

```
yuriy@prospero-lab-03:~$ sudo cat /etc/os-release
[sudo] password for yuriy:
PRETTY_NAME="Ubuntu 23.04"
NAME="Ubuntu"
VERSION_ID="23.04"
VERSION="23.04 (Lunar Lobster)"
VERSION_CODENAME=lunar
ID=ubuntu
```

Установка Minikube

Обновляем систему:

```
yuriy@prospero-lab-03:~$ sudo apt update && sudo apt upgrade -y
[sudo] password for yuriy:
Hit:1 http://packages.microsoft.com/repos/code stable InRelease
Hit:2 https://brave-browser-apt-release.s3.brave.com stable InRelease
Hit:3 http://hk.archive.ubuntu.com/ubuntu lunar InRelease
Hit:4 http://hk.archive.ubuntu.com/ubuntu lunar-updates InRelease
Hit:5 http://hk.archive.ubuntu.com/ubuntu lunar-backports InRelease
Get:6 http://hk.archive.ubuntu.com/ubuntu lunar-security InRelease [90.7 kB]
Fetched 90.7 kB in 2s (58.5 kB/s)
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
All packages are up to date.
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
Calculating upgrade... Done
0 upgraded, 0 newly installed, 0 to remove and 0 not upgraded.
yuriy@prospero-lab-03:~$
```

Устанавливаем необходимые пакеты зависимостей:

```
yuriy@prospero-lab-03:~$ sudo apt install curl
sudo apt install apt-transport-https
sudo apt install virtualbox
```

Устанавливаем kubectl:

```
yuriy@prospero-lab-03:~$ curl -LO "https://dl.k8s.io/release/${curl -L -s https://dl.k8s.io/release/stable.txt}/bin/linux/amd64/kubectl"
sudo install -o root -g root -m 0755 kubectl /usr/local/bin/kubectl

% Total    % Received % Xferd Average Speed   Time    Time     Time  Current
           Dload  Upload   Total   Spent    Left   Speed
100 138    100 138    0    0   654      0 --:--:-- --:--:-- --:--:--   657
100 46.9M 100 46.9M    0    0  9.8M      0 0:00:04 0:00:04 --:--:-- 10.7M
yuriy@prospero-lab-03:~$
```

Устанавливаем Minikube:

```
yuriy@prospero-lab-03:~$ curl -LO https://storage.googleapis.com/minikube/releases/latest/minikube-linux-amd64
sudo install minikube-linux-amd64 /usr/local/bin/minikube

% Total    % Received % Xferd Average Speed   Time    Time     Time  Current
           Dload  Upload   Total   Spent    Left   Speed
100 80.0M 100 80.0M    0    0 10.3M      0 0:00:07 0:00:07 --:--:-- 10.5M
yuriy@prospero-lab-03:~$
```

Запускаем и проверяем версию Minikube:

```
yuriy@prospero-lab-03:~$ minikube start
🐹 minikube v1.30.1 on Ubuntu 23.04
🌟 Automatically selected the virtualbox driver. Other choices: ssh, none
📀 Downloading VM boot image ...
> minikube-v1.30.1-amd64.iso: 65 B / 65 B [-----] 100.00% ? p/s 0s
> minikube-v1.30.1-amd64.iso: 282.84 MiB / 282.84 MiB 100.00% 9.77 MiB p/
👍 Starting control plane node minikube in cluster minikube
📦 Downloading Kubernetes v1.26.3 preload ...
> preloaded-images-k8s-v18-v1...: 397.02 MiB / 397.02 MiB 100.00% 10.64 M
🔥 Creating virtualbox VM (CPUs=2, Memory=2200MB, Disk=20000MB) ...
🛠 Preparing Kubernetes v1.26.3 on Docker 20.10.23 ...
   ■ Generating certificates and keys ...
   ■ Booting up control plane ...
   ■ Configuring RBAC rules ...
🔗 Configuring bridge CNI (Container Networking Interface) ...
   ■ Using image gcr.io/k8s-minikube/storage-provisioner:v5
🔍 Verifying Kubernetes components...
🌟 Enabled addons: default-storageclass, storage-provisioner
🏁 Done! kubectl is now configured to use "minikube" cluster and "default" namespace by default
yuriy@prospero-lab-03:~$ minikube version
minikube version: v1.30.1
commit: 08896fd1dc362c097c925146c4a0d0dac715ace0
```

Установка Docker

Добавляем ключ GPG Docker:

```
yuriy@prospero-lab-03:~$ curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --dearmor -o /usr/share/keyrings/docker-archive-keyring.gpg
yuriy@prospero-lab-03:~$
```

Добавление репозитория Docker:

```
yuriy@prospero-lab-03:~$ echo "deb [arch=amd64 signed-by=/usr/share/keyrings/docker-archive-keyring.gpg] https://download.docker.com/linux/ubuntu $(lsb_release -cs) stable" | sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
yuriy@prospero-lab-03:~$
```

Установка Docker Engine:

```
yuriy@prospero-lab-03:~$ sudo apt update
sudo apt install docker-ce docker-ce-cli containerd.io
```

Добавляем текущего пользователя в группу Docker и перезагружаем систему:

```
yuriy@prospero-lab-03:~$ sudo usermod -aG docker $USER
yuriy@prospero-lab-03:~$ sudo reboot
```

Проверяем версию Docker и текущее состояние:

```
yuriy@prospero-lab-03:~$ docker --version
Docker version 23.0.6, build ef23cbc
yuriy@prospero-lab-03:~$ sudo systemctl status docker
● docker.service - Docker Application Container Engine
   Loaded: loaded (/lib/systemd/system/docker.service; enabled; preset: enabled)
   Active: active (running) since Sun 2023-05-14 17:16:03 HKT; 3min 50s ago
     TriggeredBy: ● docker.socket
       Docs: https://docs.docker.com
      Main PID: 1198 (dockerd)
         Tasks: 11
        Memory: 96.0M
          CPU: 336ms
         CGroup: /system.slice/docker.service
                 └─1198 /usr/bin/dockerd -H fd:// --containerd=/run/containerd/containerd.sock
```

Создадим докер-образ для нашего веб-сервера (все оригиналы файлов в папке ".files"):

```
GNU nano 7.2 Dockerfile
FROM php:7.4-apache

LABEL Yura Shamray

# Установка YAML зависимостей
RUN apt-get update && \
    apt-get install -y libyaml-dev && \
    rm -rf /var/lib/apt/lists/*

# Установка YAML расширений
RUN pecl install yaml && \
    docker-php-ext-enable yaml

# Установка MySQL расширений
RUN docker-php-ext-install mysqli pdo pdo_mysql

# Копирование файлов в контейнер
COPY index.php /var/www/html/index.php
COPY mifiib-secret.yaml /var/www/html/mifiib-secret.yaml
COPY mifiib-config.yaml /var/www/html/mifiib-config.yaml
# Настройка прав на директорию
RUN chown -R www-data:www-data /var/www/html
```

Из файла `mifiib-config.yaml` php-скрипт будет извлекать адрес сервера баз данных сервера, а из `mifiib-secret.yaml` логин и пароль.

Создадим файл index.php:

```
GNU nano 7.2                                index.php
#!/usr/bin/php

error_reporting(E_ALL);

# Загрузка значения адреса сервера из файла mifiib-config.yaml
$configFile = 'mifiib-config.yaml';
$config = yaml_parse_file($configFile);
$databaseServer = $config['data']['mifiib-url'];

# Загрузка значений логина и пароля из файла mifiib-secret.yaml
$secretFile = 'mifiib-secret.yaml';
$secret = yaml_parse_file($secretFile);
$databaseUsername = base64_decode($secret['data']['username']);
$databasePassword = base64_decode($secret['data']['password']);

$link = mysqli_connect($databaseServer, $databaseUsername, $databasePassword, "mifiib");

if (mysqli_connect_errno()) {
    printf("Can't connect to: %s\n", mysqli_connect_error());
    exit();
}

$data = '';

if ($result = mysqli_query($link, "SELECT * FROM user_list")) {
    while ($row = $result->fetch_assoc()) {
        //echo $row['id']. ' - '. $row['name']. ' - ' . $row['surname']. "<br>";

        $data.= "<tr><td>".$row['id']. "</td><td>".$row['name']. "</td><td>".$row['surname']. "</td></tr>";
    }
}

$html = '
<style>.users { padding: 0; margin: 0 auto; border: 1px solid #000; color: #000; border-collapse: collapse; } .users th { text-align: left; } .users td { padding: 5px; font-size: 14px; }</style>
<table class="users" padding="0" cellspacing="0"><tr><th width="100">ID</th><th width="300">Name</th><th width="300">Surname</th></tr>'. $data. '</table>
';

echo $html;

phpinfo();
?>
```

Создадим докер-образ для нашего сервера баз данных:

```
GNU nano 7.2                                Dockerfile
FROM mysql:5.7
LABEL Yura Shamray
USER root
RUN chmod 755 /var/lib/mysql
ENV MYSQL_DATABASE=mifiib
COPY ./database.sql /docker-entrypoint-initdb.d/
```

Файл с таблицей для нашей базы:

```
GNU nano 7.2                                database.sql
CREATE TABLE `user_list` (
  `id` int(11) NOT NULL,
  `name` varchar(255) NOT NULL,
  `surname` varchar(255) NOT NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8;

INSERT INTO `user_list` (`id`, `name`, `surname`) VALUES
(1, 'Pancho', 'Villa'),
(2, 'Salvatore', 'Giuliano'),
(3, 'Sultana', 'Daku');

ALTER TABLE `user_list`
  ADD PRIMARY KEY (`id`);

ALTER TABLE `user_list`
  MODIFY `id` int(11) NOT NULL AUTO_INCREMENT, AUTO_INCREMENT=4;
COMMIT;
```

Создаём образы и загружаем их в свой репозиторий на Docker Hub (скриншоты из редактора VS Code):

```
yuriy@prospero-lab-03:~/mydev/php$ docker build -t yurashamray/php:7.4-apache .
[+] Building 1.0s (13/13) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 752B
=> [internal] load .dockerignore
=> => transferring context: 2B
=> [internal] load metadata for docker.io/library/php:7.4-apache
=> [1/8] FROM docker.io/library/php:7.4-apache@sha256:c9d7e608f73832673479770d66aacc8100011ec751d1905ff63fae3fe2e0ca6d
=> [internal] load build context
=> => transferring context: 106B
=> CACHED [2/8] RUN apt-get update && apt-get install -y libyaml-dev && rm -rf /var/lib/apt/lists/*
=> CACHED [3/8] RUN pecl install yaml && docker-php-ext-enable yaml
=> CACHED [4/8] RUN docker-php-ext-install mysqli pdo pdo_mysql
=> CACHED [5/8] COPY index.php /var/www/html/index.php
=> [6/8] COPY mifiib-secret.yaml /var/www/html/mifiib-secret.yaml
=> [7/8] COPY mifiib-config.yaml /var/www/html/mifiib-config.yaml
=> [8/8] RUN chown -R www-data:www-data /var/www/html
=> exporting to image
=> => exporting layers
=> => writing image sha256:27e05c9c86c86819b17af7d7cd78a4089eb138354fdc7bef09db3562ae6e7718
=> => naming to docker.io/yurashamray/php:7.4-apache
```

```
yuriy@prospero-lab-03:~/mydev/php$ docker push yurashamray/php:7.4-apache
The push refers to repository [docker.io/yurashamray/php]
0d84702d4899: Preparing
f7f4422bb15d: Preparing
f7f4422bb15d: Pushed
8b5fd3e0191f: Pushed
2530bdc5b9d0: Pushed
09129aa850e7: Pushed
147977437ec9: Pushed
3d33242bf117: Mounted from library/php
529016396883: Mounted from library/php
5464bcc3f1c2: Mounted from library/php
28192e867e79: Mounted from library/php
d173e78df32e: Mounted from library/php
0be1ec4fbfdc: Mounted from library/php
30fa0c430434: Mounted from library/php
a538c5a6e4e0: Mounted from library/php
e5d40f64dcb4: Mounted from library/php
44148371c697: Mounted from library/php
797a7c0590e0: Mounted from library/php
f60117696410: Mounted from library/php
ec4a38999118: Mounted from library/php
7.4-apache: digest: sha256:4a6448b3299ddbd63f8c1a641cb12902b8059c8ee9daa3061873a8cbf695c0f1 size: 4494
```

```
yuriy@prospero-lab-03:~/mydev/mysql$ docker build -t yurashamray/mysql:5.7 .
[+] Building 1.5s (9/9) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 187B
=> [internal] load .dockerignore
=> => transferring context: 2B
=> [internal] load metadata for docker.io/library/mysql:5.7
=> [auth] library/mysql:pull token for registry-1.docker.io
=> [1/3] FROM docker.io/library/mysql:5.7@sha256:f57eef421000aaf8332a91ab0b6c96b3c83ed2a981c29e6528b21ce10197cd16
=> [internal] load build context
=> => transferring context: 34B
=> CACHED [2/3] RUN chmod 755 /var/lib/mysql
=> CACHED [3/3] COPY ./database.sql /docker-entrypoint-initdb.d/
=> exporting to image
=> => exporting layers
=> => writing image sha256:ee0a1f77db471289196f44999f02b89bbf0fa37f0865c9836a32278793dd90d2
=> => naming to docker.io/yurashamray/mysql:5.7
```

```
yuriy@prospero-lab-03:~/mydev/mysql$ docker push yurashamray/mysql:5.7
The push refers to repository [docker.io/yurashamray/mysql]
6dd80440318e: Pushed
08563ed35eb5: Pushed
14b7cdb4d0b8: Mounted from library/mysql
cd0d43032c74: Mounted from library/mysql
a7b93ed0c3e3: Mounted from library/mysql
eb6a593247bf: Mounted from library/mysql
bfd2860a9b6a: Mounted from library/mysql
f1320287a031: Mounted from library/mysql
e792fc7474b3: Mounted from library/mysql
74a008e3042b: Mounted from library/mysql
519cf9da0b88: Mounted from library/mysql
963177f01181: Mounted from library/mysql
e74a57638021: Mounted from library/mysql
5.7: digest: sha256:cad5be3c447596208e20b98f1468ad8e59c17fa459560eebbf398aa0bac6b4a1 size: 3032
```

Создадим манифесты Kubernetes в формате YAML (все оригиналы файлов в папке “.files”):

GNU nano 7.2	mifiib-config.yaml
apiVersion: v1 kind: ConfigMap metadata: name: mifiib-config data: mifiib-url: mifiib-service	
GNU nano 7.2	mifiib-secret.yaml
apiVersion: v1 kind: Secret metadata: name: mifiib-secret type: Opaque data: username: cm9vdA== password: c2VjcmlVOMTIz	

```

GNU nano 7.2                                                                    mifiib.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: mifiib-deployment
  labels:
    app: mifiib
spec:
  replicas: 1
  selector:
    matchLabels:
      app: mifiib
  template:
    metadata:
      labels:
        app: mifiib
    spec:
      containers:
        - name: mifiibdb
          image: yurashanray/mysql:5.7
          ports:
            - containerPort: 3306
          resources:
            limits:
              cpu: "0.5"
              memory: "512Mi"
          env:
            - name: MYSQL_ROOT_USERNAME
              valueFrom:
                secretKeyRef:
                  name: mifiib-secret
                  key: username
            - name: MYSQL_ROOT_PASSWORD
              valueFrom:
                secretKeyRef:
                  name: mifiib-secret
                  key: password
            ---
apiVersion: v1
kind: Service
metadata:
  name: mifiib-service
spec:
  selector:
    app: mifiib
  ports:
    - protocol: TCP
      port: 3306
      targetPort: 3306

```

```

GNU nano 7.2                                                                    webapp.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: webapp-deployment
  labels:
    app: webapp
spec:
  replicas: 1
  selector:
    matchLabels:
      app: webapp
  template:
    metadata:
      labels:
        app: webapp
    spec:
      containers:
        - name: webapp
          image: yurashanray/php:7.4-apache
          ports:
            - containerPort: 80
          resources:
            limits:
              cpu: "0.5"
              memory: "512Mi"
          env:
            - name: USER_NAME
              valueFrom:
                secretKeyRef:
                  name: mifiib-secret
                  key: username
            - name: USER_PWD
              valueFrom:
                secretKeyRef:
                  name: mifiib-secret
                  key: password
            - name: DB_URL
              valueFrom:
                configMapKeyRef:
                  name: mifiib-config
                  key: mifiib-url
            ---
apiVersion: v1
kind: Service
metadata:
  name: webapp-service
spec:
  type: NodePort
  selector:
    app: webapp
  ports:
    - protocol: TCP
      port: 80
      targetPort: 80
      nodePort: 30080

```


Применяем манифесты Kubernetes командами:

```
kubectl apply -f mifiib-config.yaml
kubectl apply -f mifiib-secret.yaml
kubectl apply -f mifiib.yaml
kubectl apply -f webapp.yaml
```

Проверяем статус разворачивания с помощью следующей команды:

```
yuriy@prospero-lab-03:~/mydev/php$ kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
mifiib-deployment-5ff8f76988-6886k 1/1     Running   0           77m
webapp-deployment-55b7cbb488-c55kz  1/1     Running   0           77m
```

Видим наши запущенные поды.

Проверяем службы с помощью следующей команды:

```
yuriy@prospero-lab-03:~/mydev/php$ kubectl get services
NAME             TYPE        CLUSTER-IP   EXTERNAL-IP   PORT(S)          AGE
kubernetes       ClusterIP   10.96.0.1    <none>        443/TCP          85m
mifiib-service   ClusterIP   10.97.89.147 <none>        3306/TCP         79m
webapp-service   NodePort    10.104.98.134 <none>        80:30080/TCP     79m
```

Получаем IP-адрес Minikube:

```
yuriy@prospero-lab-03:~/mydev/php$ minikube ip
192.168.49.2
```

Открываем веб-браузер и перейдём по адресу <http://192.168.49.2:30080>

PHP 7.4.33 - phpinfo()

Not secure | 192.168.49.2:30080

yurashamray/... yurashamray (...)

The Students List (MIFIIB)

ID	Name	Surname
1	Pancho	Villa
2	Salvatore	Giuliano
3	Sultana	Daku

PHP Version 7.4.33

System	Linux webapp-deployment-55b7cbb488-c55kz 6.2.0-20-generic #20-Ubuntu SMP PREEMPT_DYNAMIC Thu Apr 6 07:48:48 UTC 2023 x86_64
Build Date	Nov 15 2022 06:03:12
Configure Command	"/configure" "--build=x86_64-linux-gnu" "--with-config-file-path=/usr/local/etc/php" "--with-config-file-scan-dir=/usr/local/etc/php/conf.d" "--enable-option-checking=fatal" "--with-mhash" "--with-pic" "--enable-ftp" "--enable-mbstring" "--enable-mysqlnd" "--with-password-argon2" "--with-sodium=shared" "--with-pdo-sqlite=/usr" "--with-sqlite3=/usr" "--with-curl" "--with-iconv" "--with-openssl" "--with-readline" "--with-zlib" "--disable-phpdbg" "--with-pear" "--with-libdir=lib/x86_64-linux-gnu" "--disable-cgi" "--with-apxs2" "build_alias=x86_64-linux-gnu"
Server API	Apache 2.0 Handler
Virtual Directory Support	disabled
Configuration File (php.ini) Path	/usr/local/etc/php
Loaded Configuration File	(none)
Scan this dir for additional .ini files	/usr/local/etc/php/conf.d
Additional .ini files parsed	/usr/local/etc/php/conf.d/docker-php-ext-mysqli.ini, /usr/local/etc/php/conf.d/docker-php-ext-pdo_mysql.ini, /usr/local/etc/php/conf.d/docker-php-ext-sodium.ini, /usr/local/etc/php/conf.d/docker-php-ext-yaml.ini
PHP API	20190902
PHP Extension	20190902
Zend Extension	320190902
Zend Extension Build	API320190902.NTS
PHP Extension Build	API20190902.NTS

Мы видим, что наше веб-приложение работает и отображает таблицу списка студентов из базы данных MySQL.

Инвентаризация кластера в файле `./files/cluster-info.xlsx`

Практическое задание #02 завершено.